

Anomaly Detection in Surveillance Videos

Domain: Computer Vision

Project Overview

This project involves developing an AI system capable of detecting anomalous or unusual activities in surveillance video footage. The focus is on leveraging unsupervised or semi-supervised learning techniques to identify deviations from normal behavior patterns without relying heavily on labeled anomaly data. Candidates will work with video datasets such as UCSD Pedestrian or ShanghaiTech to build models that can effectively distinguish between normal and anomalous events in crowded or complex scenes.

Business Context

Surveillance systems generate vast amounts of video data, making manual monitoring impractical and error-prone. Automated anomaly detection can enhance security by providing real-time alerts for suspicious activities, reducing response times, and optimizing resource allocation. This capability is critical in public safety, transportation hubs, and critical infrastructure monitoring, where early detection of unusual behavior can prevent incidents or mitigate risks.

Technical Objectives

- Design and implement a video processing pipeline to extract meaningful spatiotemporal features from surveillance footage.
- Develop unsupervised or semi-supervised learning models to learn normal activity patterns and detect anomalies as deviations.
- Utilize datasets such as UCSD Pedestrian or ShanghaiTech to train and validate the anomaly detection models.
- Incorporate techniques for handling imbalanced data and limited labeled anomaly examples.
- Optimize model performance for real-time or near-real-time anomaly detection scenarios.

System Architecture Guidelines

The system should include a preprocessing module for video frame extraction and normalization, followed by a feature extraction component using convolutional neural networks or other suitable architectures. Temporal modeling can be achieved through recurrent networks, autoencoders, or transformer-based models to capture motion and behavior patterns. The anomaly detection module should output anomaly scores or binary labels per frame or event. The architecture should be modular to allow experimentation with different feature extractors and anomaly detection algorithms. Efficient data loading and batch processing are recommended to handle large video

datasets.

Expected Deliverables

- A detailed project report documenting the methodology, experiments, and results.
- Source code implementing the complete anomaly detection pipeline.
- Trained models with clear instructions for inference on new surveillance videos.
- Quantitative evaluation results on the suggested datasets using defined metrics.
- Visualizations or sample outputs demonstrating detected anomalies in video sequences.

Modeling Requirements

- Employ unsupervised or semi-supervised learning approaches such as autoencoders, variational autoencoders, GANs, or clustering-based methods.
- Incorporate spatiotemporal feature extraction to capture both appearance and motion information.
- Implement anomaly scoring mechanisms that can rank or classify video segments based on abnormality likelihood.
- Address challenges related to noise, occlusion, and varying lighting conditions in surveillance footage.
- Ensure model generalization across different scenes and camera perspectives.

Evaluation Metrics

- Area Under the Receiver Operating Characteristic Curve (AUC-ROC) to measure anomaly detection performance.
- Equal Error Rate (EER) to evaluate the trade-off between false positives and false negatives.
- Precision, Recall, and F1-Score for anomaly classification accuracy.
- Frame-level and event-level detection accuracy to assess temporal localization of anomalies.
- Computational efficiency metrics such as inference time per frame.

Implementation Constraints

- Use PyTorch as the primary deep learning framework.
- Leverage Scikit-learn for auxiliary machine learning tasks and evaluation.
- Models should be trained and evaluated on publicly available datasets: UCSD Pedestrian or ShanghaiTech.
- Code must be modular, well-documented, and reproducible.
- Ensure compatibility with standard GPU hardware for training and inference.

Stretch Goals

- Incorporate attention mechanisms or transformer architectures to improve temporal modeling.
- Develop a real-time anomaly alert system with a user-friendly interface.
- Experiment with domain adaptation techniques to transfer models across different surveillance environments.
- Integrate multi-camera data fusion for enhanced anomaly detection accuracy.
- Explore explainability methods to provide interpretable anomaly detection outputs.

Skills Evaluated

- Proficiency in computer vision and video data processing.
- Expertise in unsupervised and semi-supervised machine learning techniques.
- Ability to design and implement deep learning architectures using PyTorch.
- Experience with handling imbalanced datasets and anomaly detection challenges.
- Competence in evaluating model performance using rigorous metrics.
- Capability to document and communicate technical solutions effectively.

Project Submission Format

Please follow the instructions below to ensure your project is submitted correctly and meets all requirements. Submissions not adhering to this format may not be accepted.

1. GitHub Repository Requirements

- Create a Public GitHub Repository.
- Name the repository appropriately for your project.
- Include a README.md with:
 - Project title and description
 - Setup and usage instructions
 - Dependencies or prerequisites
 - Brief explanation of your solution approach
- Regular Commits
- Commit your work frequently as you progress.
- Each commit should represent a logical, completed change.
- Write clear, descriptive commit messages.
- Avoid large, monolithic commits.
- Push to GitHub
- Ensure all required files are present and up to date.
- Do not include unnecessary files.
- Use a .gitignore file as needed.

2. Video Demonstration (Unlisted YouTube Video)

- Create a short video (3–5 minutes).
- Explain your approach.
- Demonstrate your project running.
- Optionally walk through important code sections.
- Upload the video as Unlisted.
- Copy and submit the unlisted YouTube link.

3. Submission Checklist

- GitHub repository with regular, meaningful commits.
- Complete README.md.
- All source code pushed to GitHub.
- Unlisted YouTube video link.

4. Submission Instructions

- GitHub repository URL
- Colab Notebook link
- Unlisted YouTube video link

Best Practices & Tips

- Commit often and write clear commit messages.
- Keep your repository organized and clean.
- Ensure your code runs without errors.
- Use branches for significant features.
- Double-check video visibility before submission.
- Example Commit Message: `git commit -m "Add input validation for registration form"`
- How to Set YouTube Video as Unlisted: When uploading, under "Visibility," select "Unlisted." Save and copy the video URL for submission.
- By following this format, you ensure your submission is professional, easy to review, and meets all technical requirements.